

WellnessInCloud

Relazione di Progettazione Object-Oriented

Materia: Programmazione Object Oriented · **A.A.** 2025–2026

Documento: Progettazione O-O del sistema e Diagramma delle Classi

Repository: <https://github.com/EdoksSW/WellnessInCloud.git>

1. Introduzione e obiettivi

WellnessInCloud è una soluzione gestionale per palestre e club sportivi che digitalizza l'intero rapporto con l'utente: dall'anagrafica e dal controllo degli accessi, alla pianificazione di corsi, lezioni e turni del personale, fino alla gestione delle iscrizioni e a un piccolo e-commerce di prodotti.

Questo documento descrive la **progettazione Object-Oriented** del sistema: la suddivisione in classi di dominio, le gerarchie, le enumerazioni e le scelte architetturali. La componente di base di dati (schema relazionale, trigger, SQL) è trattata a parte e non è oggetto di questa relazione.

2. Presentazione del dominio

Il dominio riguarda la gestione di una struttura sportiva in tutte le sue aree: anagrafica e documenti sanitari, logistica (corsi, lezioni, sale, turni), controllo accessi (abbonamenti e ingressi singoli) e gestione commerciale (carrello, ordini, pagamenti). I dati sono eterogenei e le regole di dominio non banali, il che ha reso naturale una modellazione a classi con ereditarietà ed enumerazioni.

2.1 Ambiti del sistema

- **Utenze e CRM:** gestione dei ruoli (Admin, Staff, Cliente) e monitoraggio delle scadenze dei certificati medici dei Clienti.
- **Planning dei corsi:** organizzazione di corsi e lezioni nelle sale, con i turni di lavoro dello staff.
- **Iscrizioni e check-in:** gestione di abbonamenti/ingressi singoli e verifica della regolarità documentale.

- **E-shop e transazioni:** carrello dei prodotti, finalizzazione degli ordini e pagamenti unificati.

3. Requisiti in linguaggio naturale

- Il sistema gestisce tre tipologie di utenza con permessi diversi: **Amministratore**, **Staff** (istruttori/receptionist) e **Clienti**.
- Ogni cliente possiede un **Certificato Medico** (con tipologia, data di emissione e di scadenza), requisito per l'accesso alla struttura.
- Il planning prevede **Corsi** collegati a **Lezioni** specifiche, svolte in **Sale** (identificate da un id); lo **Staff** è organizzato in **Turni** di lavoro.
- I clienti gestiscono le **Iscrizioni** a un **Titolo di Ingresso** (abbonamento o ingresso singolo) e possono **prenotare** le lezioni.
- È presente uno **shop**: il cliente aggiunge **Prodotti** al **Carrello** e finalizza un **Ordine**.
- Tutti i pagamenti (da iscrizione o da ordine e-commerce) confluiscono in un'unica entità **Pagamento**.

4. Funzioni ad alto livello

Modulo	Funzione	Descrizione
Gestione Utenze	Registrazione e ruoli	Creazione e gestione di profili Cliente, Staff e Admin (CRUD).
Controllo Accessi	Validazione iscrizione	Verifica di abbonamento/ingressi incrociata con la validità del certificato medico.
Logistica	Gestione spazi e turni	Assegnazione dello staff ai turni; allocazione di corsi e lezioni nelle sale.
Finance & Shop	Checkout unificato	Gestione di carrello, ordini prodotti e pagamenti (iscrizioni ed e-commerce).

5. Modello del dominio (classi principali)

Le entità sono suddivise in macro-aree (package). Per massimizzare riusabilità e semplicità di manutenzione; si è fatto ricorso all'**ereditarietà** per la gerarchia degli utenti e alle **enumerazioni** per la gestione rigorosa degli stati.

A. Profilazione e gerarchia degli utenti (`model.utenti`)

Il sistema adotta una struttura gerarchica basata sulla classe astratta `Utente`, radice comune che centralizza **tutta l'anagrafica**: codice fiscale (chiave), nome, cognome, e-mail, telefono, password, data di nascita, età, via, civico, CAP e carta fedeltà. Da essa derivano tre specializzazioni:

- `Cliente`: destinatario dei servizi; aggiunge lo **stato account** (`StatoAccount`: `ATTIVO`, `BLOCCATO`, `IN_REVISIONE` – attivo ma privo di certificato valido).
- `Staff`: dipendente della palestra; definito da **qualifica**, **IBAN** e da un **ruolo** (`RuoloStaff`: `ISTRUTTORE`, `RECEPTIONIST`).
- `Admin`: supervisore con privilegi elevati (pianificazione corsi, gestione prezzario); non aggiunge attributi propri.

B. Infrastruttura logistica e planning (`model.logistica`)

- `Corso`: attività astratta (es. "Yoga", "Spinning") con nome, descrizione e un **istruttore** (Staff) che lo tiene.
- `Lezione`: istanza temporale concreta del corso, con giorno, orario di inizio e fine; si svolge in una **sala** (riferita tramite `id_sala`) e appartiene a un corso.
- `Turno`: fascia lavorativa (data, ora inizio/fine) assegnata allo staff per coordinare il personale.
- `Prenotazione`: registra l'iscrizione di un cliente a una lezione; il suo stato (`StatoPrenotazione`) può essere `IN_ATTESA`, `CONFERMATA` o `ANNULLATA`.

C. Controllo accessi e tipologie di servizio (`model.commerce`)

Il `CertificatoMedico` (tipo, data di emissione, data di scadenza, percorso del file) è il requisito vincolante per l'accesso. L'entità `Iscrizione` collega un cliente a un `TitoloIngresso`, con data di inizio e fine. Il `TitoloIngresso` unifica le modalità di accesso tramite l'attributo **tipo** (es. Giornaliero, Mensile, Trimestrale, Annuale) e il **prezzo**; la durata dell'abbonamento viene indicata al momento dell'iscrizione (da cui si calcola la data di fine).

D. Modulo e-commerce e vendite (`model.commerce`)

I prodotti sono catalogati per `Categoria` e gestiti nel `Carrello` del cliente, che mantiene la **mappa prodotto→quantità** (`Map<Prodotto,Integer>`) e il totale. Nel momento dell'acquisto il sistema genera un `Ordine`, di cui monitora lo stato (`StatoOrdine`: `IN_ELABORAZIONE`, `PAGATO`, `SPEDITO`, `CONSEGNATO`, `ANNULLATO`) e le righe (`OrdineDettaglio`: prodotto + quantità). L'entità `Pagamento` (importo, metodo) tratta in modo uniforme le transazioni provenienti sia dalle **iscrizioni** sia dagli **ordini**, centralizzando ogni movimento economico.

Enumerazioni di stato. Gli stati del dominio sono modellati con `enum` (`StatoAccount` , `RuoloStaff` , `StatoOrdine` , `StatoPrenotazione`): garantiscono valori chiusi e leggibili, evitando l'uso di stringhe "magiche" nel codice.

6. Diagramma delle classi (UML)

Il diagramma seguente rappresenta il dominio del problema: le classi (con attributi e metodi), la gerarchia `Utente`, le enumerazioni. Le operazioni di accesso ai dati definite dai DAO sono riportate come metodi delle rispettive classi di dominio.

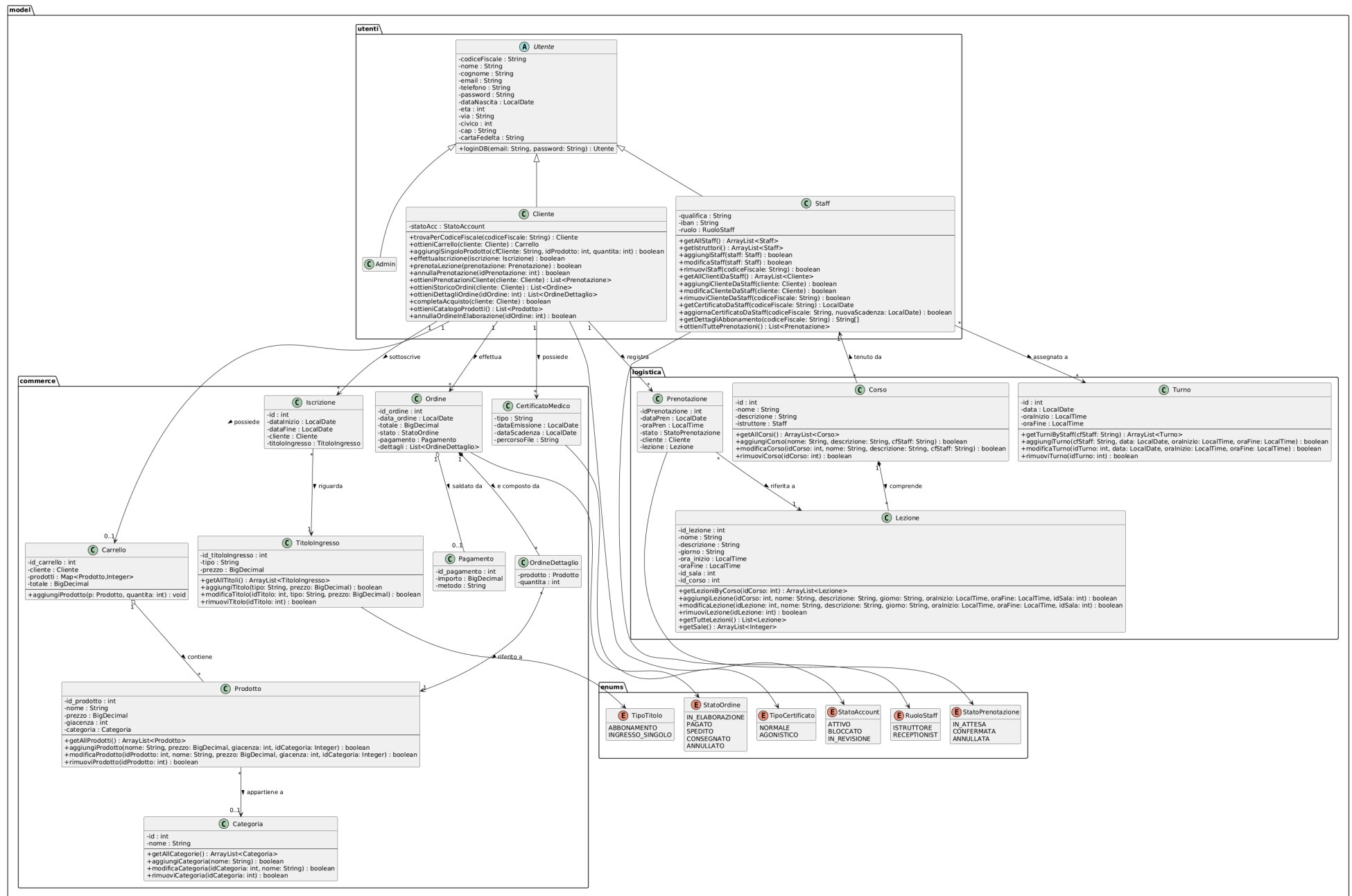


Figura 1 – Diagramma delle classi del dominio (package: utenti, commerce, logistica, enums).

6.1 Class diagram strutturale

Per una visione d'insieme, il class diagram seguente riporta le sole classi ed enumerazioni con le relazioni tra esse (ereditarietà, associazioni, aggregazioni/composizioni, molteplicità e verso di lettura), senza attributi né metodi.

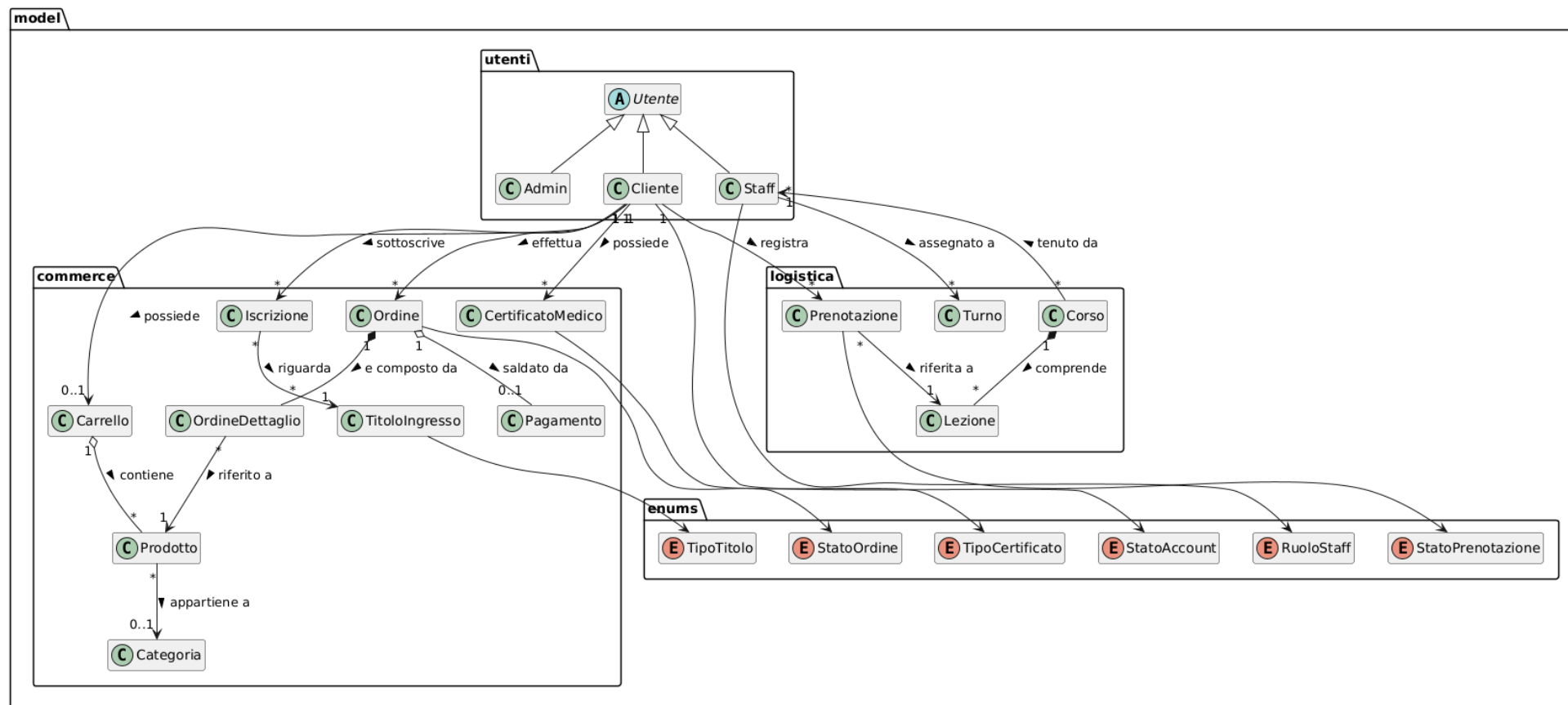


Figura 2 – Class diagram strutturale (sole classi ed enumerazioni con le relazioni).

7. Scelte progettuali

Il sistema è sviluppato in **Java** seguendo il pattern **BCE (Boundary-Control-Entity)** affiancato dal pattern **DAO (Data Access Object)** per la persistenza su database relazionale (PostgreSQL). La gestione del ciclo di vita e delle dipendenze è affidata ad **Apache Maven** (`pom.xml`).

- **Boundary** (package `gui`): interfacce Swing realizzate con i file `.form` ; leggono l'input e mostrano l'output, delegando ogni logica al Controller.
- **Control** (`Controller`): oggetto unico, creato all'avvio e passato a ogni schermata; coordina Model e DAO.
- **Entity** (package `model`): gli oggetti di dominio descritti sopra.
- **DAO**: le operazioni di persistenza sono definite in **interfacce** (package `dao`) e realizzate da implementazioni PostgreSQL, così da poter sostituire la tecnologia di persistenza senza impattare gli altri livelli.

Principi OO applicati: incapsulamento (attributi privati con accessori), **ereditarietà** e **generalizzazione** (gerarchia `Utente` → Admin/Cliente/Staff), **polimorfismo** nel trattamento uniforme degli utenti, uso di **collezioni generiche** (mappe e liste) e di **enumerazioni** per gli stati.